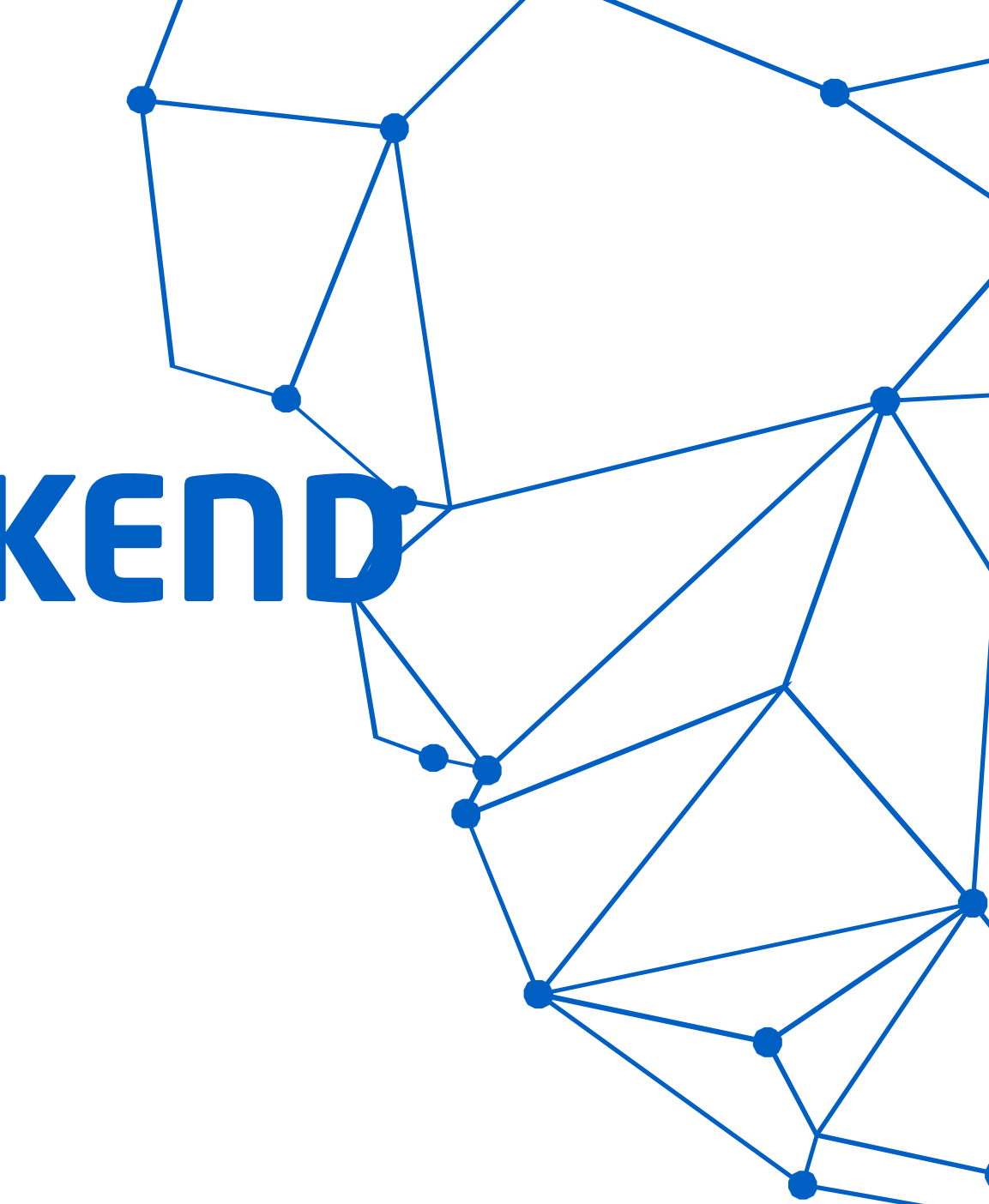
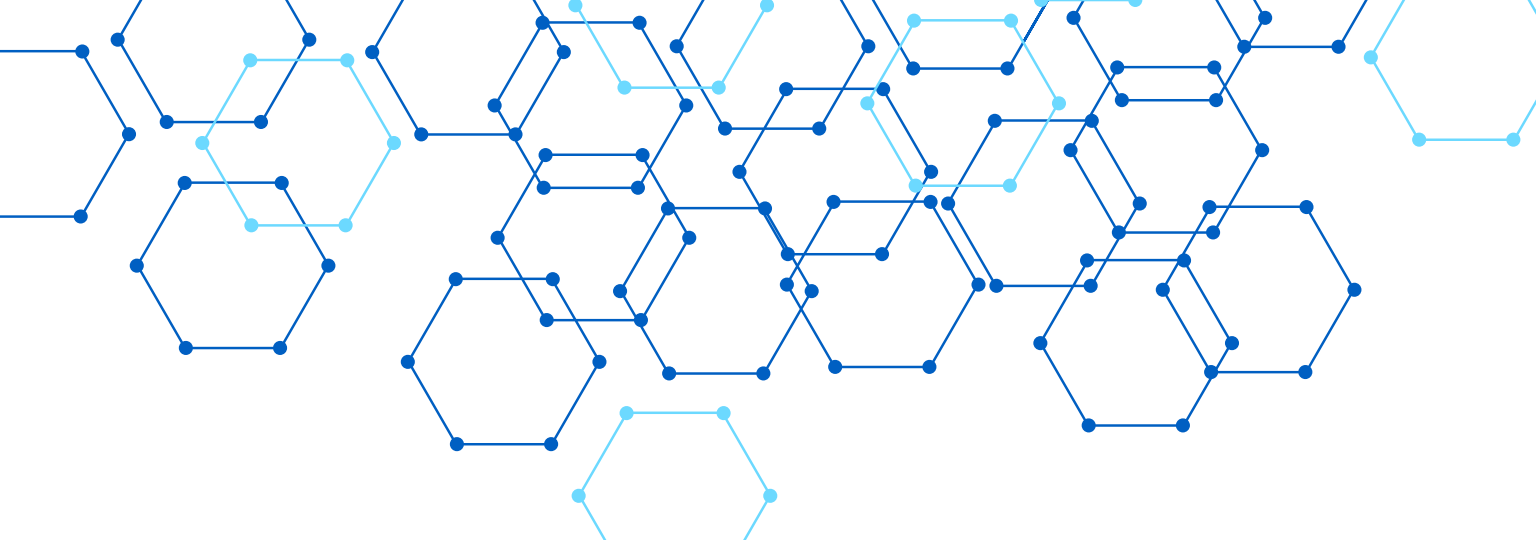
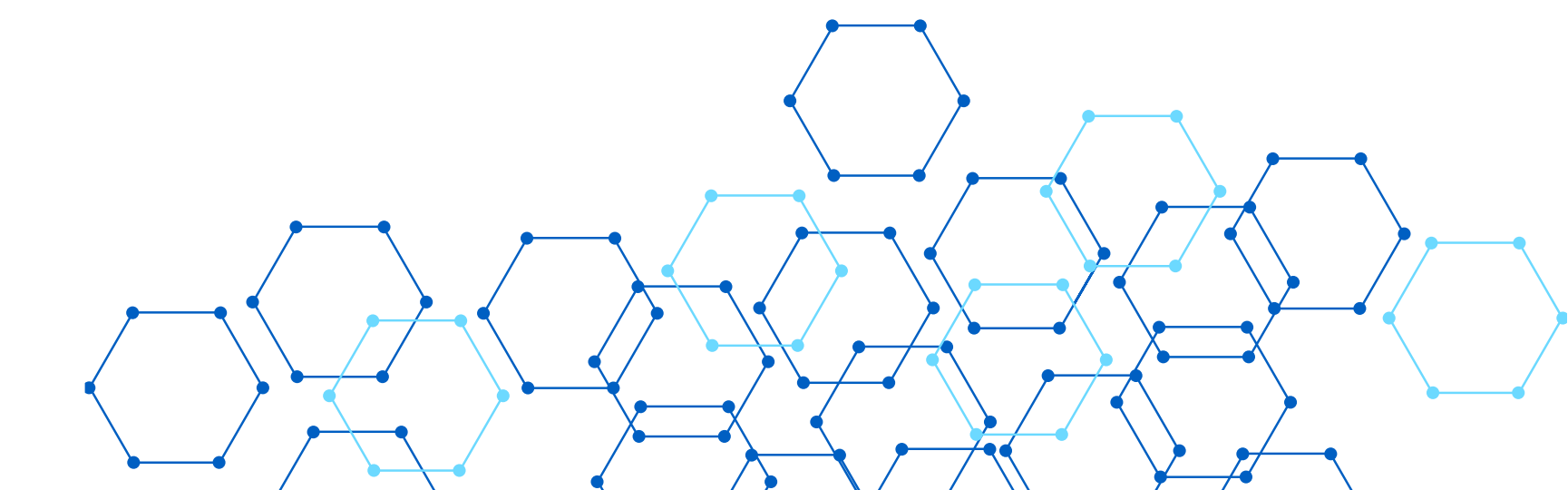


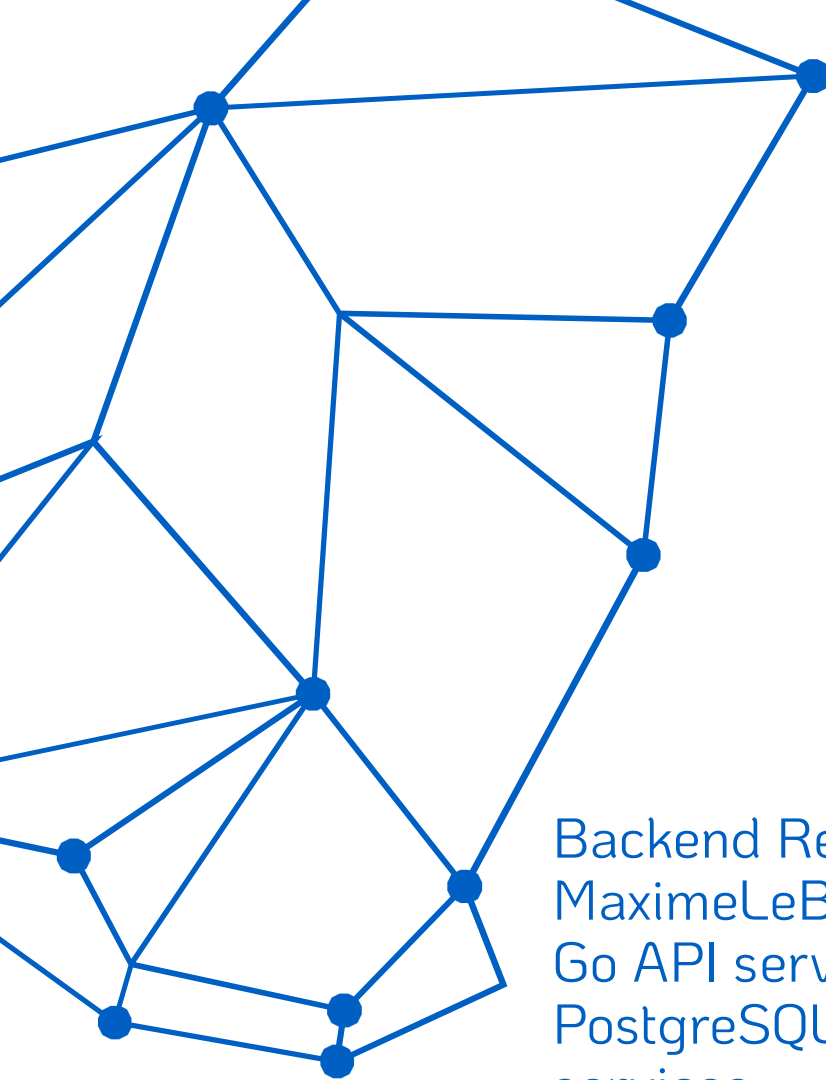


# FUSIONFEST BACKEND & FRONTEND ARCHITECTURE

How the Go Server and Flutter Mobile App Work Together



# PROJECT OVERVIEW



Backend Repository  
MaximeLeBesnerais/FusionFest-Server  
Go API server with chi router and  
PostgreSQL database for all core  
services.

Purpose  
Backend serves the REST API and  
static admin UI. Frontend provides  
the mobile experience for festival  
attendees with offline support.

Frontend Repository  
MaximeLeBesnerais/fusion\_fest  
Flutter mobile app with Dart SDK,  
Riverpod state management, and  
offline-first architecture.



# BACKEND ARCHITECTURE



Tech Stack: Go 1.25 with go-chi/chi/v5 HTTP router for lightweight, composable routing. PostgreSQL database via jackc/pgx/v5 driver for robust data persistence. Email OTP authentication with mock or Resend provider integration. Static web UI served directly from the Go server.



Key Features: User management with email OTP verification flow. Points system with event ledger for tracking all point transactions. QR code scanning for check-ins and interactions. Trivia questions with timed rotation windows. Hype voting system. Referral system with activation logic. Rewards wallet with token-based redemption. Rate limiting and HMAC authentication.

# FRONTEND ARCHITECTURE



Tech Stack: Flutter with Dart SDK ^3.11.3, Riverpod for state management, GoRouter for navigation, Dio for HTTP requests, flutter\_secure\_storage for credentials, mobile\_scanner for QR codes, connectivity\_plus for offline detection, shared\_preferences for offline queue.



Key Features: Email OTP onboarding flow, HMAC request signing for secure API calls, QR code scanner integration, Trivia participation, Hype voting, Leaderboard display, Rewards wallet with QR redemption, Referral code sharing, Offline queue with automatic sync on reconnection.

# AUTHENTICATION FLOW



Steps 1-3: User enters email → Server sends OTP via Resend/mock → User verifies OTP code → Server validates and creates user record in database.



Steps 4-6: Server generates unique device\_id and secret\_key → Returns credentials to mobile app → App stores securely via flutter\_secure\_storage.



Steps 7-8: HMAC signature format: METHOD|path|timestamp → Headers: X-Device-ID, X-Timestamp, X-Signature (HMAC-SHA256 of message with secret\_key).



# QR SCAN FLOW



Scan: Mobile app uses mobile\_scanner to capture QR codes. Camera activates, detects QR pattern, extracts the qrId from the code data for API submission.



Validate: POST /qr/{qrId}/scan with HMAC auth headers (X-Device-ID, X-Timestamp, X-Signature). Server validates festival window, QR active status, and prevents duplicate scans.



Record: Server records scan in qr\_scans table and creates point\_events entry. Mobile receives response and displays success confirmation or error message to user.



# TRIVIA FLOW



Fetch: App calls GET /trivia/current to retrieve the currently active trivia question. Server returns question text, options, and time window for the active question.



Answer: User selects response and app sends POST /trivia/answer with question ID and selected option. Request includes HMAC authentication headers for validation.

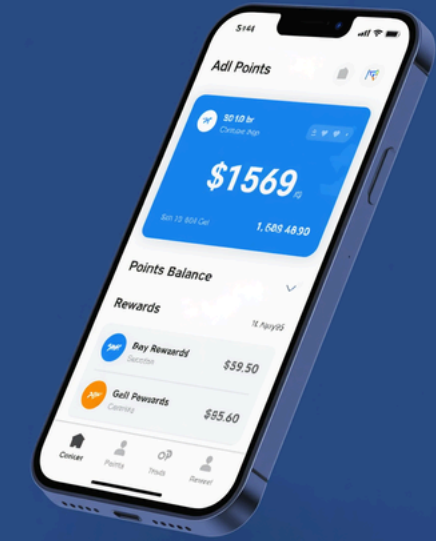


Validate: Server checks question status, verifies user hasn't answered, confirms festival is active. Records trivia\_answers entry and awards points if correct. Returns result to mobile.

# REWARDS FLOW



Wallet: GET /rewards/wallet returns user's current points balance, leaderboard rank, and available rewards catalog. Mobile displays wallet UI with claimable items based on point thresholds.



Claim: POST /rewards/claim validates points, stock availability, and no active claims. Server creates reward\_claims record with unique redeem token. Mobile generates QR: fusionfest://redeem/{token}



Redeem: Staff scans user's QR code and calls POST /staff/rewards/redeem with token. Server validates token, marks claim as redeemed with timestamp. User receives confirmation of successful redemption.





# OFFLINE SYNC FLOW



Queue: Actions stored in SharedPreferences (max 100 entries) when device is offline. Each action includes timestamp, type, and payload for later processing.



Detect: connectivity\_plus package monitors network status changes. Automatically triggers sync when connection is restored.



Sync: SyncService calls POST /points/sync with queued events. Server validates, mobile clears synced entries, invalidates points and leaderboard cache.



# API CONTRACT & CONFIGURATION

## API Route Mapping

- PointsService → /points, /leaderboard
- QRService → /qr/{id}/scan
- TriviaService → /trivia/current, /trivia/answer



Configuration Alignment  
Mobile uses  
AppConfig.baseUrl set at  
build time via --dart-define  
flag. Server must be  
publicly accessible at  
matching URL for all API  
calls.

## Additional Services

- HypeService → /hype/\*
- WalletService → /rewards/wallet
- ReferralService → /referral/\*
- IdentityService → /identity/\*



# DATABASE SCHEMA HIGHLIGHTS



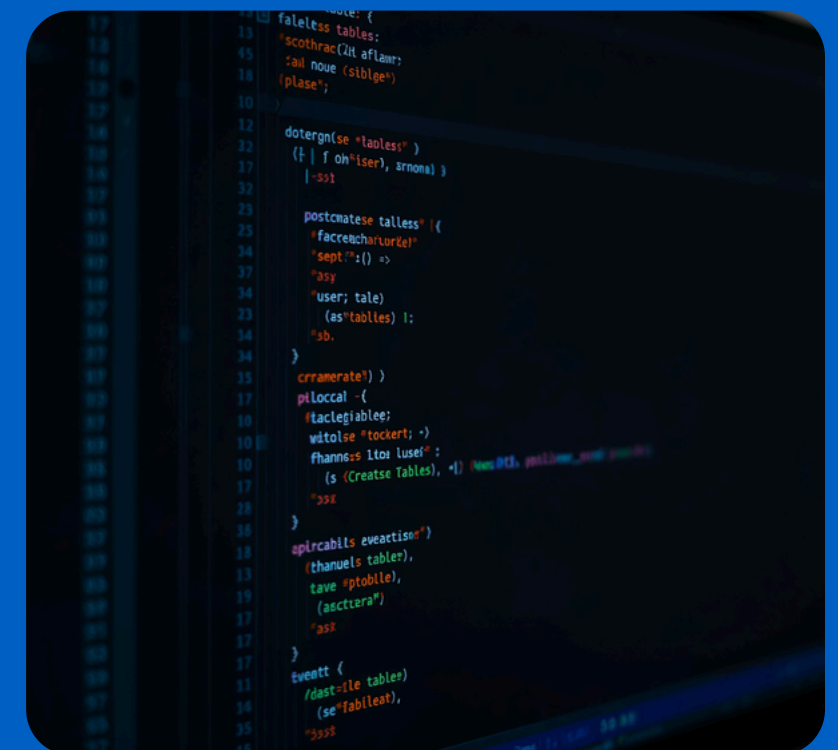
Core Tables: users (verified accounts), pending\_users (OTP verification queue), point\_events (immutable ledger tracking all point transactions with event types and timestamps)



Feature Tables: qr\_codes & qr\_scans (double-scan prevention), trivia\_questions & trivia\_answers (rotation windows), hype\_sets & hype\_votes (one vote per user constraint)



Rewards Tables: referrals & referral\_pending (activation logic), rewards & reward\_claims (token-based redemption), events (general activity logging for auditing)



# RATE LIMITING



Global IP Limit: 60 requests/min  
User Routes: 30 requests/min per X-Device-ID  
Applies to all authenticated endpoints

## Specific POST Limits:

- /points/sync - stricter throttling
- /qr/{id}/scan - prevent spam scanning
- /trivia/answer - one answer per window
- /hype/vote - single vote enforcement
- /rewards/claim - controlled redemption rate

# REFERRAL SYSTEM



## Generate Referral Code

POST /referral/generate returns referral\_code + install\_url. Users share the link with friends. New users sign up with the code, creating a referral\_pending entry that activates on their first participation event.

## Referral Rewards

- +100 points for the referrer when their referral activates.
- +25 points for the referred user as a welcome bonus.
- +250 bonus points awarded when a user reaches 3 successful referrals.



# DEPLOYMENT & APK DISTRIBUTION



Build: GitHub Actions compiles APK with `--dart-define=API_BASE_URL` configuration, embedding the production server endpoint directly into the Flutter application binary.



Publish: Compiled APK is deployed to VPS filesystem, making it available for direct download without app store dependencies or review processes.



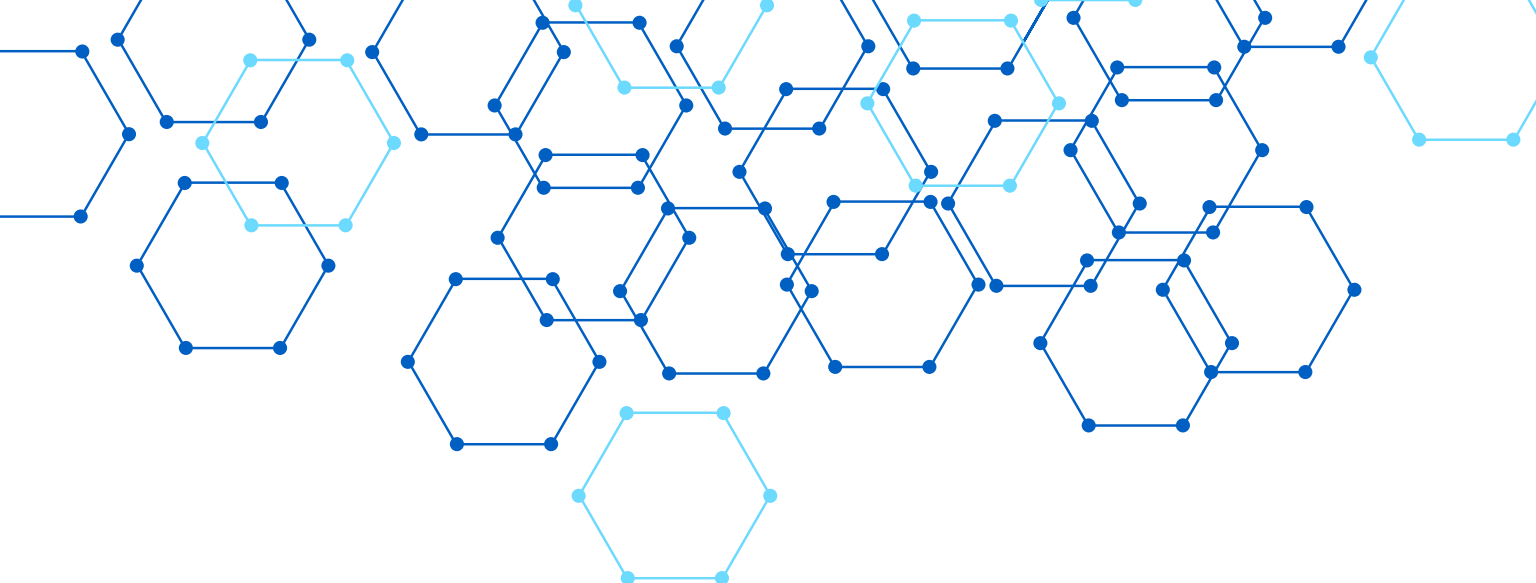
Distribute: Server exposes `GET /downloads/fusionfest-android-latest.apk` for APK retrieval and `GET /downloads/version.json` for version checking and update notifications.



# SYSTEM ARCHITECTURE

fusion\_fest Flutter App → Dio HTTP with HMAC Headers → FusionFest-Server (Go + chi + PostgreSQL) → PostgreSQL Database. Server also serves Static Web UI and APK Downloads. Core integration: Mobile app signs all requests with HMAC-SHA256, server validates and processes against PostgreSQL.





# THANK YOU

Questions? Let's discuss the architecture!

